

Distributed LLM Fine-Tuning & Inference on HPC Systems



Norwegian research infrastructure services

Hicham Agueny, PhD
Scientific Computing Group
Info.Tech. Department, UiB/NRIS



SSH setup to Olivia

```
$ssh username@olivia.sigma2.no
```

```
Out: (hicham@login.olivia.sigma2.no) One-time password (OATH) for `hicham':
```

```
Out: (hicham@login.olivia.sigma2.no) Password:
```

```
$mkdir -p /cluster/work/projects/nn9970k/$USER
```

```
$cd /cluster/work/projects/nn9970k/$USER
```

```
$git clone https://github.com/HichamAgueny/llm-hpc-course.git
```

```
$cd llm-hpc-course
```

```
$bash setup_env.sh
```

Challenges with Fine-Tuning LLMs

Storage & Memory Bottlenecks but also cost

- Kimi-2.5 .1T (FP16): checkpoint $\approx$ **2 TB (20TB for 10 tasks)**
- GPU memory requirement (FP16 – 2 bytes): **$\sim$ 6 TB**
- GPU-hour costs ($\sim$ 15–35 NOK/hour)

What is Needed?

- **High-Performance Computing (HPC)** with:
 - Powerful CPUs & GPUs
 - High-speed interconnects
- **Well-optimized software stack including e.g.**
 - Communication libraries
 - ML frameworks ...etc
- **Optimization strategies** to overcome hardware limitations

Why HPC Matters

- Without HPC systems, fine-tuning large LLMs (e.g., Kimi-2.5 1T) on large datasets is nearly impossible.



Purpose of this Course

- Introduce **fine-tuning concepts & optimization strategies**
- Provide **hands-on practice**
- Demonstrate **HPC-specific execution**



Optimization Strategy and Implementation

Addressing Compute and Memory Constraints

- **LoRA (Low-Rank Adaptation):** Lower compute cost; GPU memory remains a challenge.
- **QLoRA (Quantized LoRA):** Lower memory footprint (4-bit); additional computation overhead.
- **QAT (Quantization-Aware Training):** Balances memory, compute efficiency, and model accuracy.

Technology Stack

- **Training & Optimization:** PyTorch 2.8 (CUDA 13.0), TorchTune, TorchAO, BitsAndBytes
- **Inference & Serving:** vLLM 0.12.0 (container image) & 0.11.0 (module)
- **Software Environment:** Apptainer (Singularity), EESSI (experimental)
- **Target Platform:** Olivia Supercomputer (Cray architecture, HPE Slingshot interconnect, Arm CPUs, NVIDIA GH200)

Training Configurations

- Single GPU
- Multi-GPU (single node)
- Multi-GPU (multi-node)



Day 1 —

Single-GPU Fine-Tuning & HPC Foundations

10:00 – 11:00 | HPC Foundations for LLM Workloads (Olivia)

- Overview of Olivia Supercomputer
- Setting up container environment - **interactive session**

11:15 – 12:00 | Introduction to Fine-Tuning & Optimization Strategies

- Overview of LLM and Fine-tuning
- Concepts: LoRA, LoRA Dropout, QLoRA & QAT-LoRA

Hands-On Session – Practical Fine-Tuning on a single GPU

Task: Summarization (Xsum dataset) & QA (Alpaca dataset)

13:00 – 13:30 | Config file deep dive

13:30 – 14:15 | LoRA fine-tuning workflow

14:30 – 15:15 | Quantized fine-tuning with QLoRA

15:15 – 15:45 | Profiling

15:45 – 16:00 | Wrap-up & Discussion

Day 2 —

Distributed Training & Optimized Inference

09:30 – 10:15 | Distributed Training Concepts

- DDP vs FSDP

Hands-On Session – Practical Multi-GPU Fine-Tuning & Inference & Serving

Multi-GPU Fine-Tuning (Alpaca dataset)

10:30 – 11:15 | Multi-GPU single node: LoRA, QLoRA, QAT-LoRA

11:15 – 12:00 | Multi-GPU multi-node: LoRA, QLoRA, QAT-LoRA

Inference & Serving

13:00 – 13:30 | Introduction to the vLLM inference engine

13:30 – 14:15 | Single-GPU Inference & Serving (LoRA, QLoRA, PTQ & QAT-LoRA)

14:30 – 15:30 | Multi-GPU Inference & Serving (LoRA, QLoRA & PTQ & QAT-LoRA)

15:30 – 16:00 | Wrap-up & Discussion

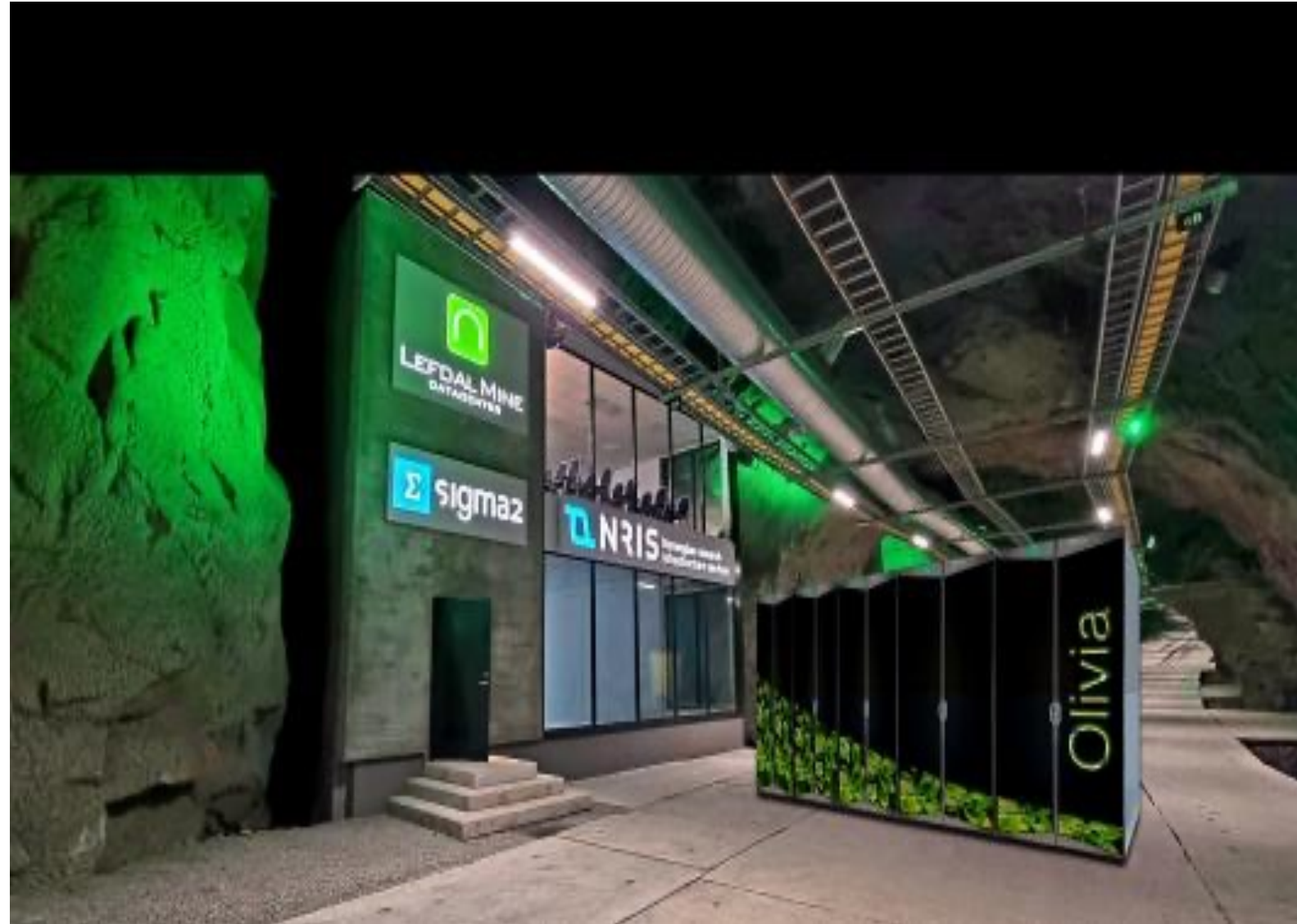
HPC Foundations for LLM Workloads

- Overview of Olivia supercomputer (CPU, GPU, Storage)
- Containerized environment: PyTorch, Torchao, TorchTune
- Verifying **CUDA support** and GPU availability

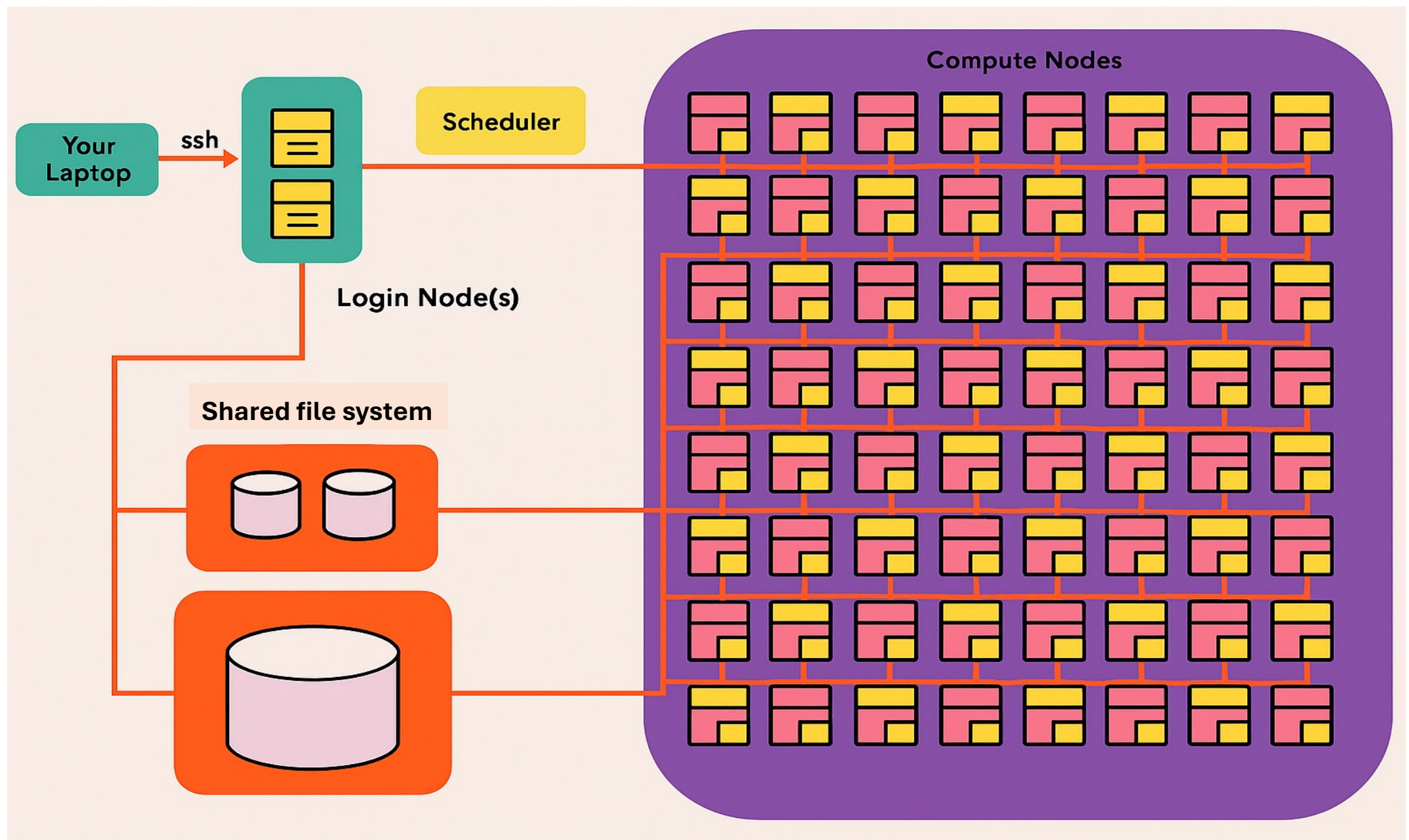


Norway's New Supercomputer - Olivia

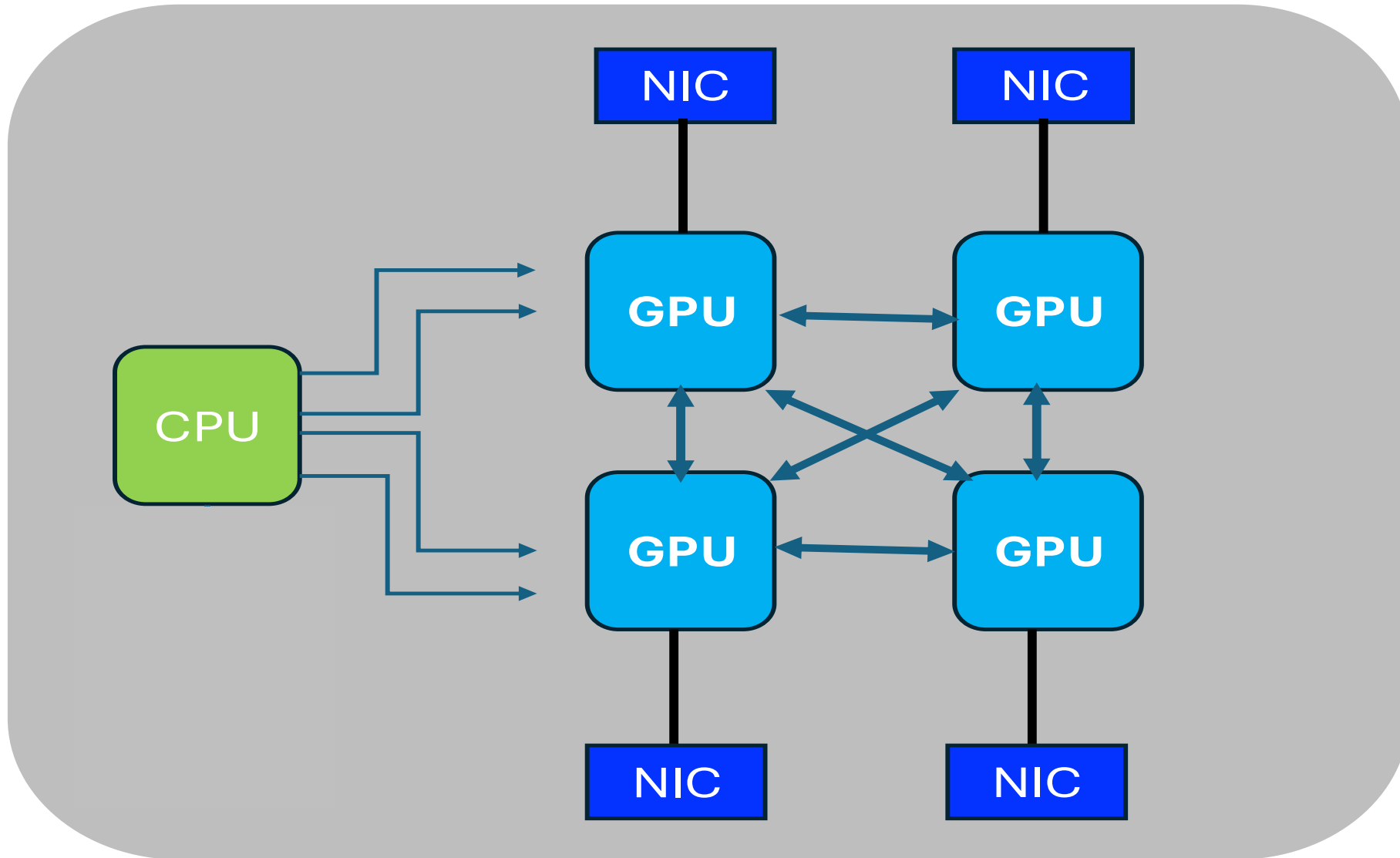
- **Olivia** is located in Lefdal Mine (Kjølsdalen)
- System: **HPE Cray Supercomputing EX**
- The GPU partition consists of:
 - 112 GPU nodes (**448 GPUs**)
 - **NVIDIA Grace Hopper Superchip**
(**GH200 96 GB**)
 - **HPE Slingshot Interconnect**



Basic Cluster-Architecture



Compute node diagram (Simplified)



Overview of the Grace Hopper System

Olivia:
Each GH200 device has 96 GB HBM

NVIDIA GH200 Grace Hopper Superchip

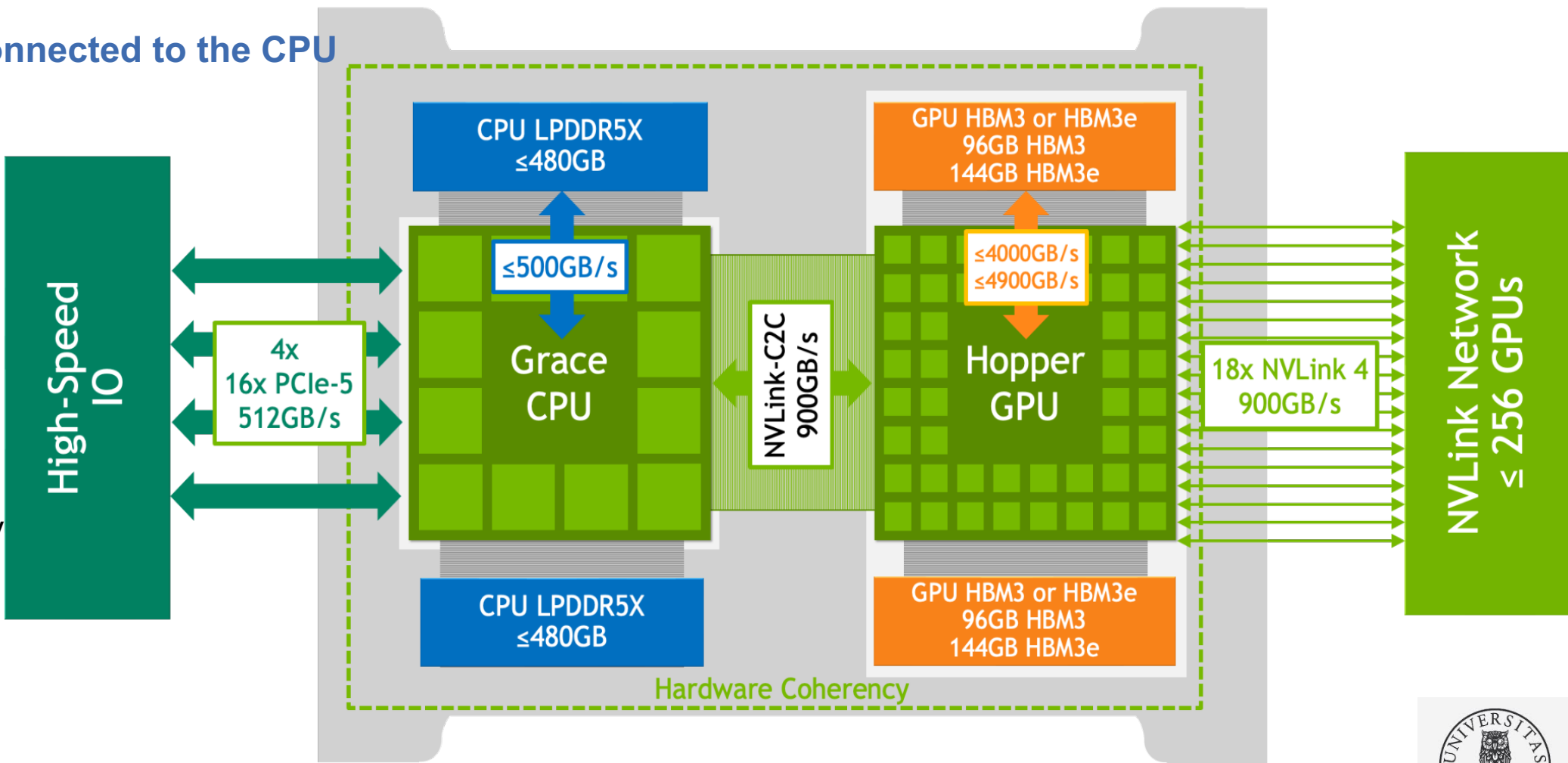
120 GB LPDDR5x connected to the CPU

72 cores per socket

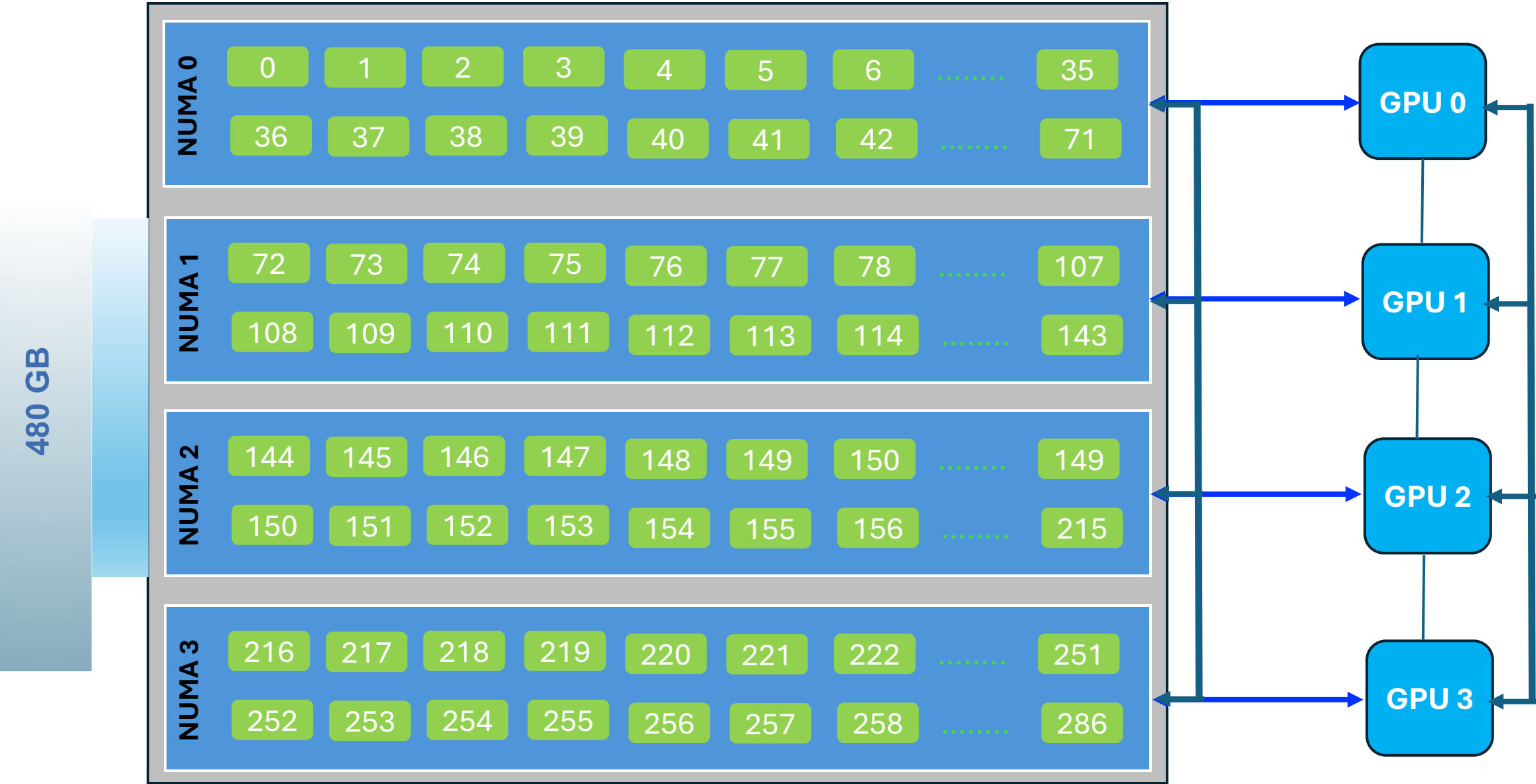
NVLink-C2C:
900GB/s of
coherent memory:

CPU and GPU threads
can concurrently
access both
CPU and GPU memory

Total of 864 GB



CPU-to-GPU affinity - (simplified diagram)



CPU-to-GPU affinity: Olivia system

- Binding a process or thread to a specific GPU.
- Workload is consistently executed on the specified GPU(s).
- Improve performance by reducing data transfer overhead.

```
hicham@x1000c0s1b1n0:~> nvidia-smi topo --matrix
GPU0      GPU1      GPU2      GPU3      CPU Affinity  NUMA Affinity  GPU NUMA ID
GPU0      X         NV6       NV6       NV6       0-71    0           4
GPU1      NV6       X         NV6       NV6       72-143   1          12
GPU2      NV6       NV6       X         NV6       144-215  2          20
GPU3      NV6       NV6       NV6       X         216-287  3          28
```

GPU	CPU Affinity	NUMA Affinity	
GPU0	0-71	0	
GPU1	72-143	1	
GPU2	144-215	2	
GPU3	216-287	3	

`--cpu-bind=map_cpu:...` → explicitly binds each task to the CPU cores nearest its GPU.

SLURM will then match **rank 0** → **GPU0 (cores 0-71)**, **rank 1** → **GPU1 (cores 72-143)**, ... & so on.

File System Storage

Divide storage into several filesystems: **/cluster**; **/scratch**; **/flash**
Spaces can be physically or logically separated

```
hicham@uan02:~> df -H -t lustre | awk '{sub(/.*:\/, "/", $1); printf "%-25s %8s %8s %8s %8s %8s\n", $1, $2, $3, $4, $5, $6}'
```

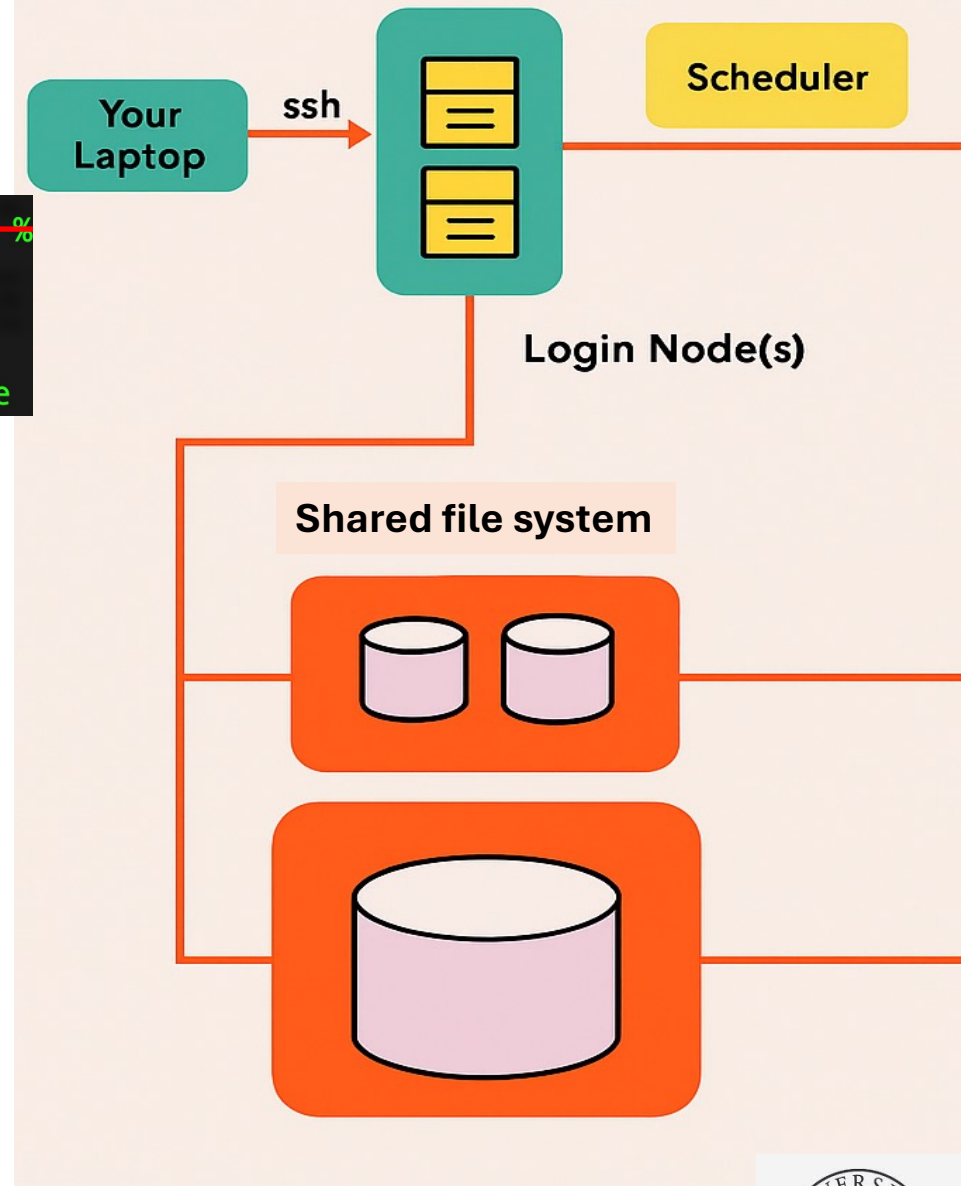
Filesystem	Size	Used	Avail	Use%	Mounted
/cluster	3.2P	39T	3.2P	2%	/cluster
/scratch	1.2P	128T	1.1P	12%	/cluster/work
/flash	524T	384G	519T	1%	/cluster/software

Persistent Project Storage: /cluster

- **Purpose:** Shared, long-term files for your entire project lifecycle.
- **Policy:** Data is **meant to stay** until the project ends.

High-Speed Scratch: /scratch

- **Purpose:** **Temporary workspace** for active jobs (high-throughput I/O).
- **Policy:** Files are **not backed up** and are **automatically deleted** after a variable time.
- **Warning:** Expect files to be purged sooner if the system is full.
Do not store critical files here.



Environment Setup on HPC - Olivia

Environment Setup on HPC - Olivia

Contents of the CUDA DL container

This container image contains the complete source of the version of PyTorch in `/opt/pytorch`. It is prebuilt and installed in the default Python environment (`/usr/local/lib/python3.10/dist-packages/torch`) in the container image.

PyTorch apptainer Container

The container also includes the following:

- Ubuntu 24.04 including Python 3.12
- NVIDIA CUDA 12.9.0
- NVIDIA cuBLAS 12.9.0
- NVIDIA cuDNN 9.10.1.3
- NVIDIA NCCL 2.26.5
- rdma-core 50.0
- NVIDIA HPC-X 2.23
- GDRCopy 2.4.1
- TensorBoard 2.16.2
- Nsight Compute 2025.2.0.11
- Nsight Systems 2025.3.1.90
- NVIDIA TensorRT™ 10.10.0.31

Pull Singularity container

```
$ apptainer pull --arch arm64 --disable-cache \
    pytorch_25.08_cuda13.0_base_arm.sif \
    docker://nvcr.io/nvidia/pytorch:25.08-py3
```

The flag “**--arch**” specifies the architecture “**arm64**”

The flag “**--disable-cache**” forces a clean and fresh download.

Definition file - Olivia

Customizing the PyTorch Container

A **Singularity definition file** is a recipe for building a container.

It specifies:

- **Bootstrap source** (base image)
- **%post** section for setup commands
- Optional metadata, environment variables, and file additions

```
Bootstrap: localimage
```

```
# Base image: Local Singularity image
```

```
From: /cluster/work/projects/nn9970k/hicham/llm-  
hpc-course/apptainer/  
pytorch_25.08_cuda13.0_base_arm.sif
```

```
%post
```

```
# Create /opt/python with symlinks for easy Python access.
```

```
mkdir -p /opt/python
```

```
ln -s /usr/bin/python3.12 /opt/python/python
```

```
# Install additional Python packages inside container
```

```
python -m pip install torchtune==0.6.1
```

```
$ apptainer build --ignore-fakeroot-command pytorch_25.08_cuda13.0_arm_custom.sif \  
pytorch2.10_cuda13.0.2_py3.12_arm.def
```

<https://md.sigma2.no/olivia-sw#Build-modified-container>

Libraries for LLM Training & Optimization

- **TorchTune:** PyTorch library for building, fine-tuning, and experimenting with LLMs. It provides well-defined **recipes** and **configurations**.
- **TorchAO:** PyTorch library for model optimizations (quantization, **memory efficiency**) to boost performance and fit larger models on hardware.



Hands-on (20 min):

A customized PyTorch container is available at your work area
`/cluster/work/projects/nn9970k/$USER/llm-hpc-course/apptainer`

- Instructions for building customized PyTorch singularity container: [llm-hpc-course/apptainer/README.md](#)

[llm-hpc-course](#) / [apptainer](#) / README.md



HichamAgueny update readme

b595ef6 · 7 minutes ago History

Preview

Code

Blame



Raw



Building a Custom Singularity Container for PyTorch

This guide explains how to build a custom PyTorch container using **Apptainer (formerly Singularity)** on the Olivia supercomputer.

Steps



Environment Setup on HPC - Olivia

Run singularity image

- Allocate an interactive session on the GPU node (**accel partition; Olivia**):

```
$srun -A NN9970K -t 00:30:00 -p accel -N 1 --gpus 1 --mem-per-cpu 8G --pty bash -i
```

- Launch PyTorch container

```
$bash launch_shell.sh
```

- Check CUDA support and other configs.

```
apptainer> From here run python  
python -c "import torch; print(torch.cuda.is_available())"  
python -c "import torch; print(torch.__config__.show())"
```

- Check packages

```
apptainer> pip list  
  
apptainer> tune ls
```


Fine-Tuning Setup

What is needed ? Download a Llama model, **Copy** Config & Recipe files

➤ After launching PyTorch container

```
Apptainer>
```

➤ Download **LLaMA 3.2-1B-Instruct** via e.g. Hugging Face

```
$tune download meta-llama/Llama-3.2-1B-Instruct  
--output-dir /cluster/work/projects/nn9970k/$USER/llm-hpc-course/data/Llama-3.2-1B-Instruct  
--ignore-patterns "original/consolidated*"  
--hf-token your-hugging-face-token
```

➤ Copy **CONFIG** file

```
$tune cp llama3_2/1B_lora_single_device .  
$tune cp llama3_2/1B_lora_multi_device .  
$tune cp generation .  
Change path: /tmp/Llama-3.2-1B-Instruct  
To: /new-Path  
sed -i "s|/tmp|/new-Path|g" 1B_lora_single_device.yaml
```

- Fine-tuning on a single GPU
- Fine-tuning on multiple GPUs
- Inference

➤ Copy built-in **RECIPE** file (python script)

```
$tune cp lora_finetune_single_device .  
$tune cp lora_finetune_distributed .  
$tune cp generate .
```

- Fine-tuning on a single GPU
- Fine-tuning on multiple GPUs
- Inference

Understanding Container Isolation vs. Bind Mounts

Test 1: run `./launch_isolated.sh`

Purpose: Demonstrates container isolation. **No bind mounts** are added (except default home).

Test Inside Container:

`$ls /workspace`

`$ls $MyWD`

`$ls $PROJECT_DIR`

Test 2: run `./demo_workspace_mount.sh`

Purpose: Shows how to make your personal files available at the **path** (`/workspace`) inside the container.

Test Inside Container:

`$ls /workspace`

`$ls $PROJECT_DIR`

Test 3: run `./demo_project_mount.sh`

Purpose: Shows how to access **shared data** at its original absolute path (useful for shared datasets).

Test Inside Container:

`$ls $PROJECT_DIR`

`$ls /workspace`

Test 4: run `./launch_shell.sh`

Test Inside Container:

`$ls $PROJECT_DIR`

`$ls /workspace`

`$ls $MyWD`



Introduction to Fine-Tuning & Optimization Strategies

- Overview of LLM and Fine-tuning
- Concepts:
 - LoRA
 - LoRA Dropout
 - QLoRA
 - QAT-LoRA

Overview of Large Language Models (LLMs)

- **LLM** is a type of AI model trained on vast amounts of **text data** to understand and generate **human-like language**.
- **LLM scope** includes tasks such as:
 - Text generation
 - **Text summarization**
 - **Question Answering (QA)**
 - Translation
 - Code generation
- **Fine-tuning** is the process of:
 - Adapting a pre-trained model (like Llama)
 - Using **task-specific or domain-specific data**
 - Continuing training on a smaller, targeted dataset
 - Updating all the parameters of the pre-trained model.
- **Why Fine-Tuning ?** It enables:
 - Faster training (without re-training the full model)
 - Better performance on domain-specific tasks (or specific datasets)
 - Reduced data requirements



Challenges of Full Fine-tuning LLM

Full fine-tuning (updating all parameters for each task) becomes expensive.

- **High GPU Memory:** Requires immense GPU memory
(e.g., **6TB for Kimi.2.5 1T** – trainable parameters)
- **High Storage Cost:** Storing instances of fine-tuned models for each task requires **massive storage**
(e.g., 10 fine-tuned Kimi.2.5 models would be **10TB (size of checkpoint)x10= 10TB**).
- **High computational cost for training:** Require significant compute resources from days to weeks depending on the model size and dataset scale.

Need of optimizing fine-tuning LLM



Introducing LoRA (Low-Rank Adaptation)

LoRA Overview

- **LoRA** is an optimization technique to reduce **compute and storage costs** of full fine-tuning.

(e.g., LoRA cuts compute by 10x)

- **Core Idea:** Freeze the pre-trained model weights (the main model's weights do not change during fine-tuning)
Inject trainable rank decomposition matrices into each layer of the Transformer architecture.
- **Concept:** Only **train low rank matrices A & B**, while keeping the **pre-trained weight matrix frozen**.

LoRA <https://arxiv.org/pdf/2106.09685>

Modified forward pass

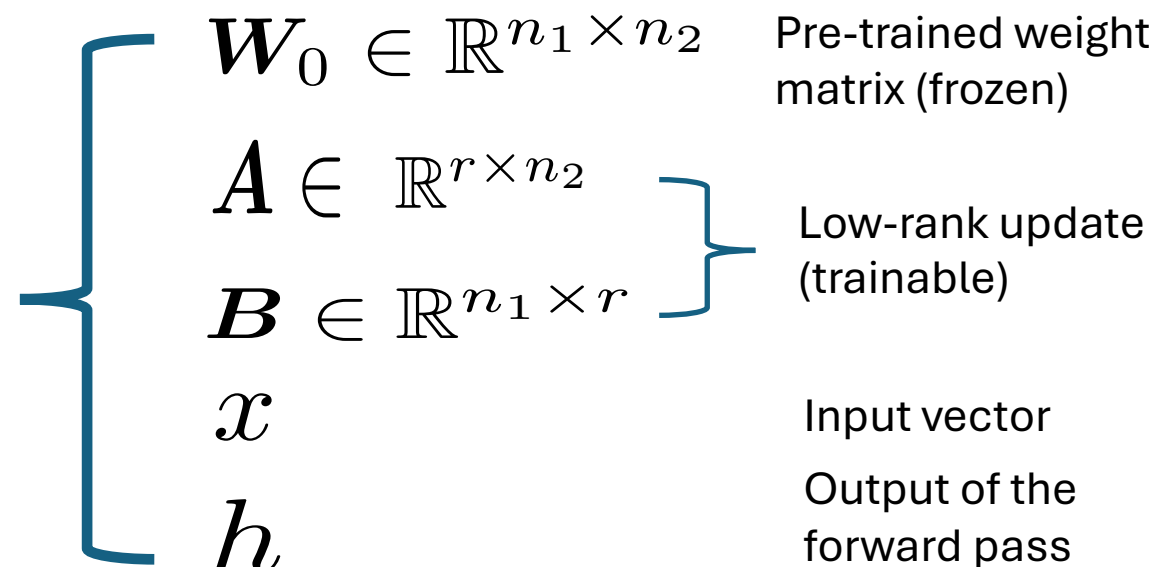
$$h = W_0 x + \Delta W x = W_0 x + B A x.$$

$$\Delta \tilde{W} = BA \quad \text{Low rank decomposition}$$

$BAx \rightarrow$ **new task behavior learned during fine-tuning**

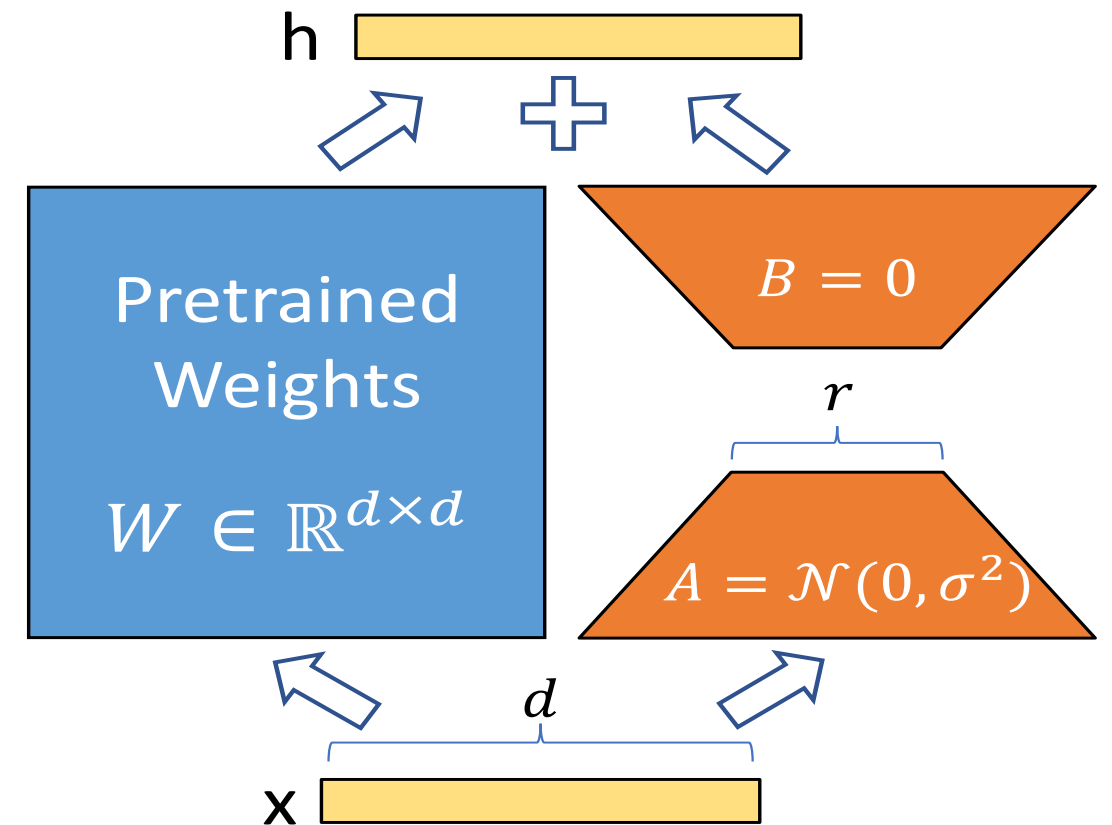
LoRA: LOW-RANK ADAPTATION OF LARGE LANGUAGE MODELS

Edward Hu* Yelong Shen* Phillip Wallis Zeyuan Allen-Zhu
Yuanzhi Li Shean Wang Lu Wang Weizhu Chen
Microsoft Corporation
{edwardhu, yeshe, phwallis, zeyuana,
yuanzhil, swang, luw, wzchen}@microsoft.com
yuanzhil@andrew.cmu.edu
(Version 2)



LoRA's Efficiency: example

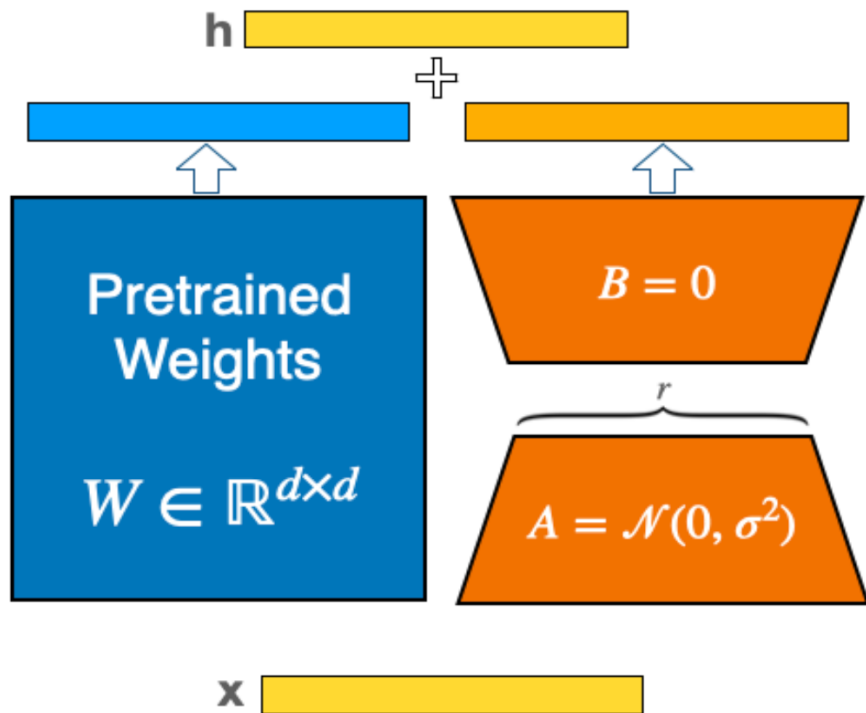
- **Full Fine-tuning:** A pre-trained weight matrix **W0** with a dimension of **800x800** has **640,000** trainable parameters.
- **LoRA Fine-tuning:** With a **rank of 8**, the number of trainable parameters is reduced to $8 \times 800(\mathbf{A}) + 800 \times 8(\mathbf{B}) = \mathbf{12,800}$.
- **Factor of Reduction:** The number of trainable parameters is reduced by a factor of **50 times** ($640,000 / 12,800 = 50$).
- **Benefit:** This significant reduction lowers both **computational** and **storage costs**.
Making it feasible to fine-tune massive models within a reasonable time.



Merging LoRA weights into the base model

- **Model merging** can be done either after the training or during the inference.

During the training



$$h = Wx + BAx$$

$$h = \underbrace{(W + BA)}_{W_{merged}}x$$

After training

Merged Weights

$$W_{merged} \in \mathbb{R}^{d \times d}$$

8B model (each element is 4 bytes (fp32)) $\rightarrow$ 32GB

10 tasks $\rightarrow$ 320 GB of storage

Total number of parameters saved is $d \times d$

LoRA Weights

$$A \in \mathbb{R}^{r \times d}$$

$$B \in \mathbb{R}^{d \times r}$$

10 tasks $\rightarrow$ 32 + 0.5 (LoRA weight) GB of storage

Total number of parameters saved is $2dr$

Advantages of LoRA

- **Massive Parameter Reduction:** Reduces trainable parameters significantly
(e.g., **10,000 times reduction** for GPT-3 175B compared to fine-tuning).
- **Reduced GPU Memory:** Lowers GPU memory requirements
(e.g., **3 times less VRAM** for GPT-3 175B, from 1.2TB to 350GB).
- **Reduced storage cost:** e.g. GPT-3 175B **reduce checkpoint size from 350 GB to 35 MB** for LoRA weights.
- **Improved Training Throughput:** Faster training due to fewer parameters needing gradient calculation and optimizer states.
(e.g., **25% speedup** on GPT-3 175B).
- **Efficient Task Switching:** **Rapidly switch between tasks** by simply swapping the small LoRA matrices, instead of reloading the entire model.
- **No Inference Latency:** Trainable LoRA matrices (A and B) can be merged with the frozen pre-trained weights.



Why LoRA seems to work ?

- **Low intrinsic rank hypothesis:** Actual update required to adapt to a new task lies in a much smaller subspace.
No need for updating the pre-trained weights.
Key information is captured even with low ranks.
- **Empirical experiments:** Tests show that even very small LoRA ranks can match full fine-tuning performance.

Model&Method	# Trainable Parameters	WikiSQL	MNLI-m	SAMSum
		Acc. (%)	Acc. (%)	R1/R2/RL
GPT-3 (FT)	175,255.8M	73.8	89.5	52.0/28.0/44.5
GPT-3 (LoRA)	4.7M	73.4	91.7	53.8/29.8/45.9
GPT-3 (LoRA)	37.7M	74.0	91.6	53.4/29.2/45.1

Risk of fine-tuning using LoRA

LoRA requires optimization of low rank for optimal performance.



Risk of overfitting on the training dataset

(affects the effectiveness of the fine-tuned model)

LoRA Dropout is a mechanism designed to overcome overfitting in LoRA-based methods

LoRA Dropout as a Sparsity Regularizer for Overfitting Control

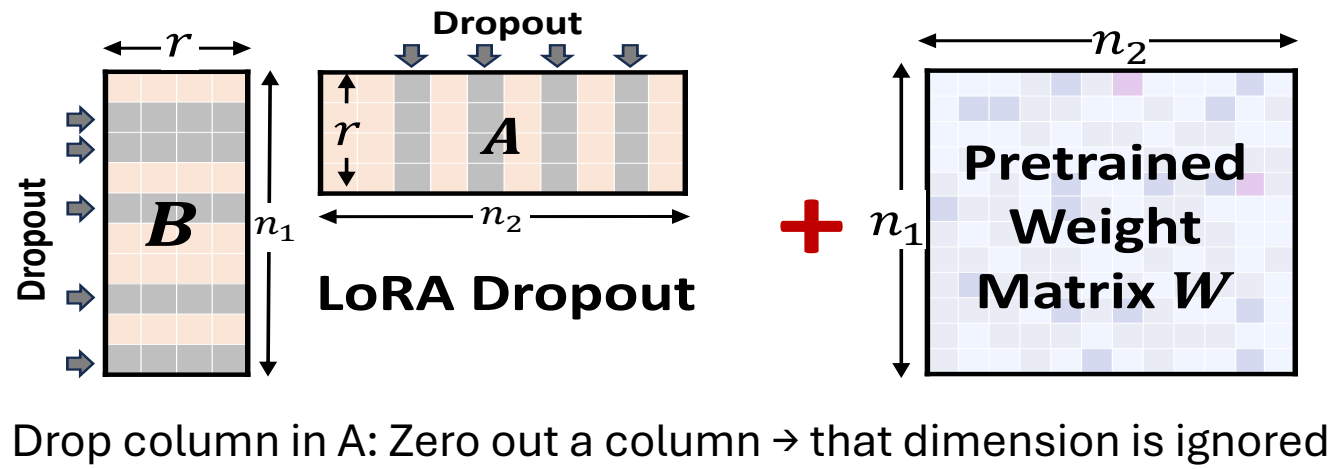
Yang Lin^{1 2 *} **Xinyu Ma**^{1 2 *} **Xu Chu**^{1 2} **Yujie Jin**^{1 2} **Zhibang Yang**² **Yasha Wang**^{1 2} **Hong Mei**¹



Concept of LoRA Dropout – for Overfitting control

Randomly drop rows and columns from both tunable low-rank parameter matrices **A** & **B**:

$$\hat{A} = A \cdot \text{diag}(m_A), m_A \sim \text{Bern}(1 - p);$$
$$\hat{B} = (B^\top \cdot \text{diag}(m_B))^\top, m_B \sim \text{Bern}(1 - p),$$



$$A \cdot \text{diag}(m_A) = A \cdot I = A$$

- **p=0**, no dropout, which means the matrices A and B are unchanged.
- **p=1**, "dropping" all the information contained in **A and B**. (**A & B are zeros - No LoRA training**)
- **p=0.1**, means that **10% of the rows in matrix B and 10% of the columns in matrix A will be randomly dropped** (set to zero) during each training step.

Method	#Params	MNLI M-Acc	SST-2 Acc	CoLA Mcc	QQP Acc	QNLI Acc	RTE Acc	MRPC Acc	STS-B Corr	All Avg.
Full Fine-Tuning	184M	89.90	95.63	69.19	92.40	94.03	83.75	89.46	91.60	88.25
LoRA _{r=8}	1.33M	90.65	94.95	69.82	91.99	93.87	85.20	89.95	91.60	88.50
LoRA+Dropout	1.33M	90.85	95.87	71.32	92.22	94.56	88.09	91.42	92.00	89.54



Other optimization strategies

- LoRA reduces training compute, but GPU memory remains a bottleneck.
- **Large models don't fit into a GPU memory.**
- DNNs traditionally use 32-bit floating-point (FP32) for weights and activations.
- Optimization **to overcome GPU limitations and speed up inference.**



Quantization

What is Quantization ?

- **Quantization** is the process of converting a model's numbers (weights and activations) from a high-precision format e.g **BF16/FP16** or **FP32** into a lower-precision format, typically 8-bit or 4-bit.
- **Ex.** In FP16: 1.23456 $\rightarrow$ 1.23 (FP8) or 123(INT8)

If **10B model** is stored in FP16/BF16, each parameter occupies 2 bytes  **~20 GB** (10B parameters * 2 bytes/parameter).

By quantizing the model to FP8, the memory required for the model weights is reduced to **~10GB**.

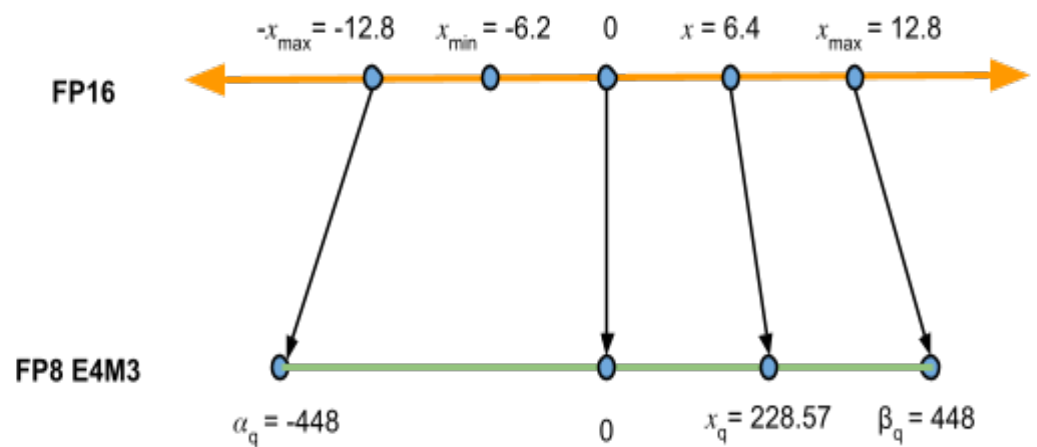
Goal:

- Quantize the model's weights to reduce its memory footprint (VRAM).
- Quantizing the model's intermediate activations (compute) can further enhance inference speed.
- Lower hardware costs



Example of Quantization algorithm

FP16 to FP8 symmetric quantization with **AbsMax** algorithm



Symmetric Quantization

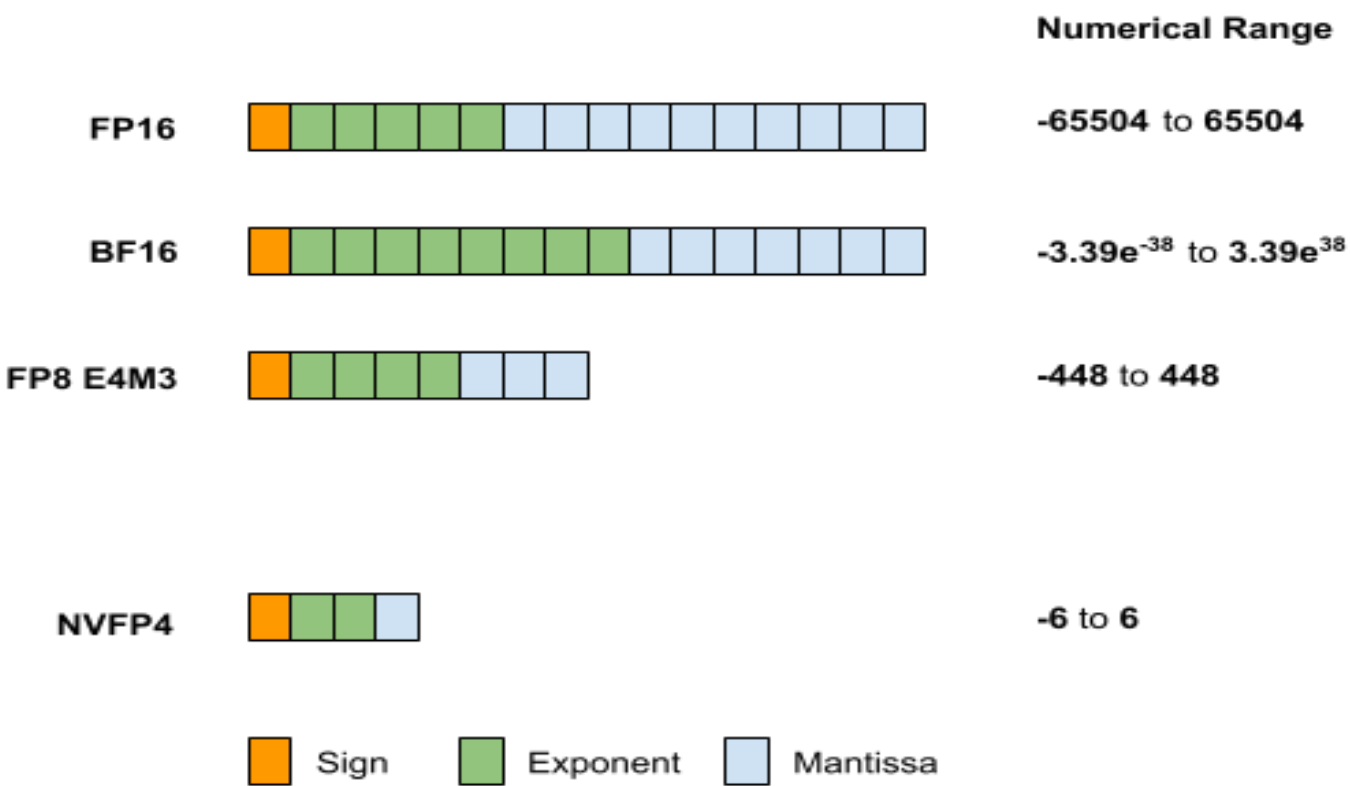
Quantization and dequantization formulas:

$$x_q = \text{clip} \left(\text{round} \left(\frac{x}{s} \right), \alpha_q, \beta_q \right)$$

α_q, β_q define the range for the quantized representations

$$x_q = \text{clip} \left(\text{round} \left(\frac{6.4}{0.028} \right), -448, 448 \right) = 228.57$$

Different floating-point representations for: FP16, BF16, FP8, FP4



Quantization approaches:

- **PTQ – Post training quantization**
- **QLoRA – Quantized LoRA**
- **QAT - Quantization aware training**

PTQ – Post training quantization

- **PTQ** is a static optimization approach
- **PTQ** converts a pre-trained floating-point model to integer precision without additional training cycles.
- **PTQ** is a process designed for preparing a model for inference.

Examples: TorchAO APIs:

Int8WeightOnlyConfig:

Weights are compressed to INT8 while activations remain BF16 (A16W8).
Ideal for memory-bound scenarios.

Int8DynamicActivationInt8WeightConfig:

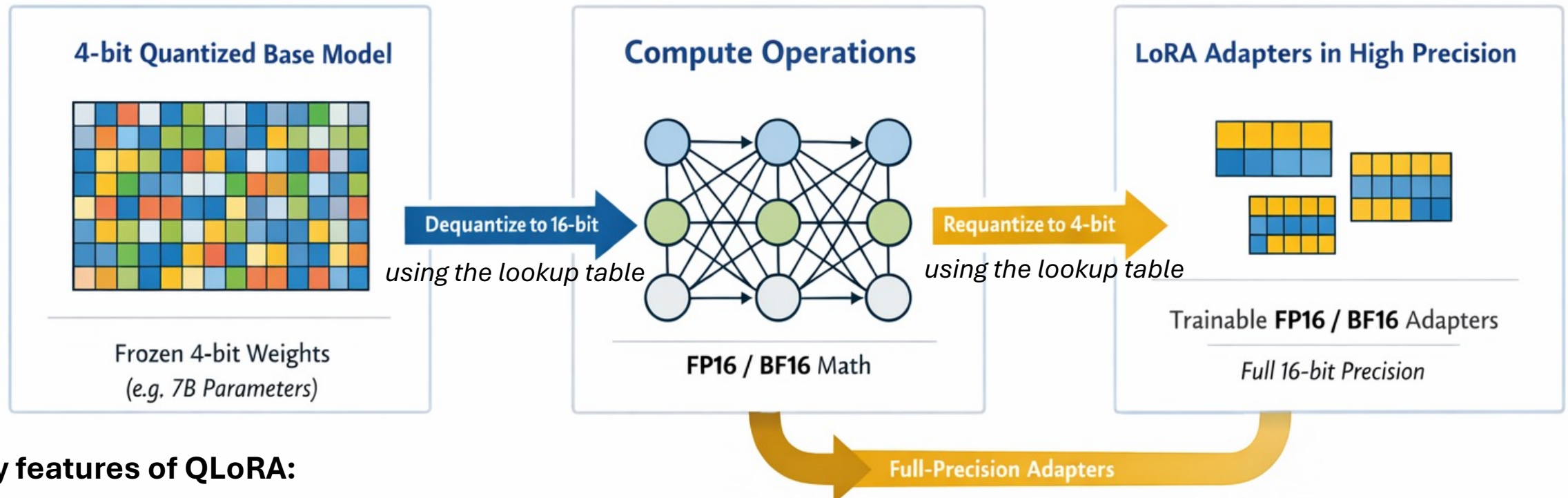
Both weights and activations are quantized to INT8 (A8W8) dynamically.
Best for compute-bound throughput.

- Because the model is never retrained, it cannot "adapt" to the rounding errors introduced by the 8-bit grid.
- PTQ causes accuracy degradation.

Concept of QLoRA – Quantized LoRA

*“QLoRA is an efficient finetuning approach that reduces memory usage enough to finetune a **65B** parameter model on a single **48GB** GPU while preserving full 16-bit finetuning task performance.”*

<https://arxiv.org/abs/2305.14314>



Key features of QLoRA:

- **4-bit NormalFloat Quantization:** Storage in NF4 (normal distribution) instead of traditional 4-bit (linear-spacing).
- **Paged optimizers:** NVIDIA unified memory feature to prevent out-of-memory.
- **Trade-off: storage vs computation**



Concept of QAT - Quantization aware training

- **QAT** is a dynamical optimization approach
- **QAT** simulates quantization during fine-tuning using "**Fake Quantization**".
- Introduces the quantization errors as part of the training loss.
- The optimizer tries to minimize the loss during the training.

Goal:
Simulates quantization effects during the training process, allowing the model to adapt to precision loss.

Diagram of a “Fake Quantization” on weights during QAT

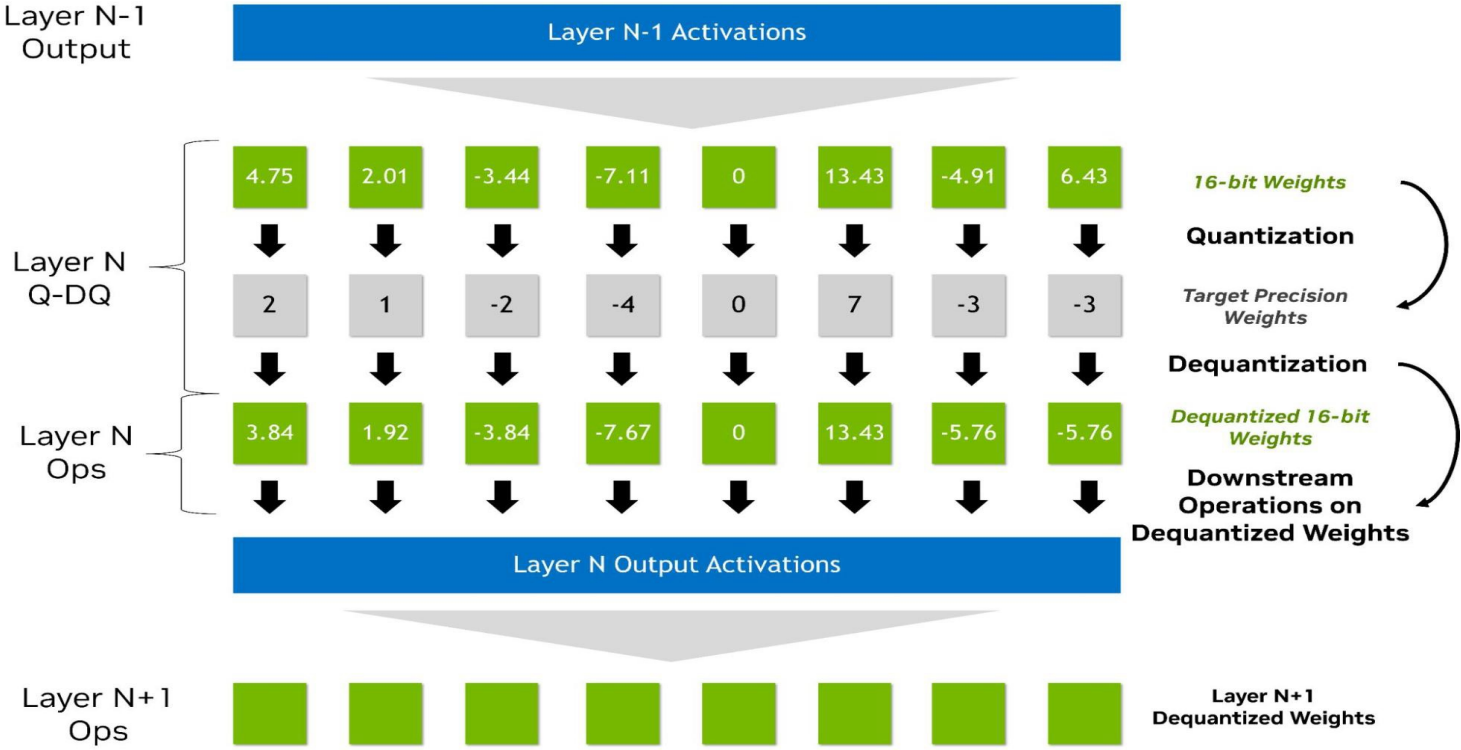
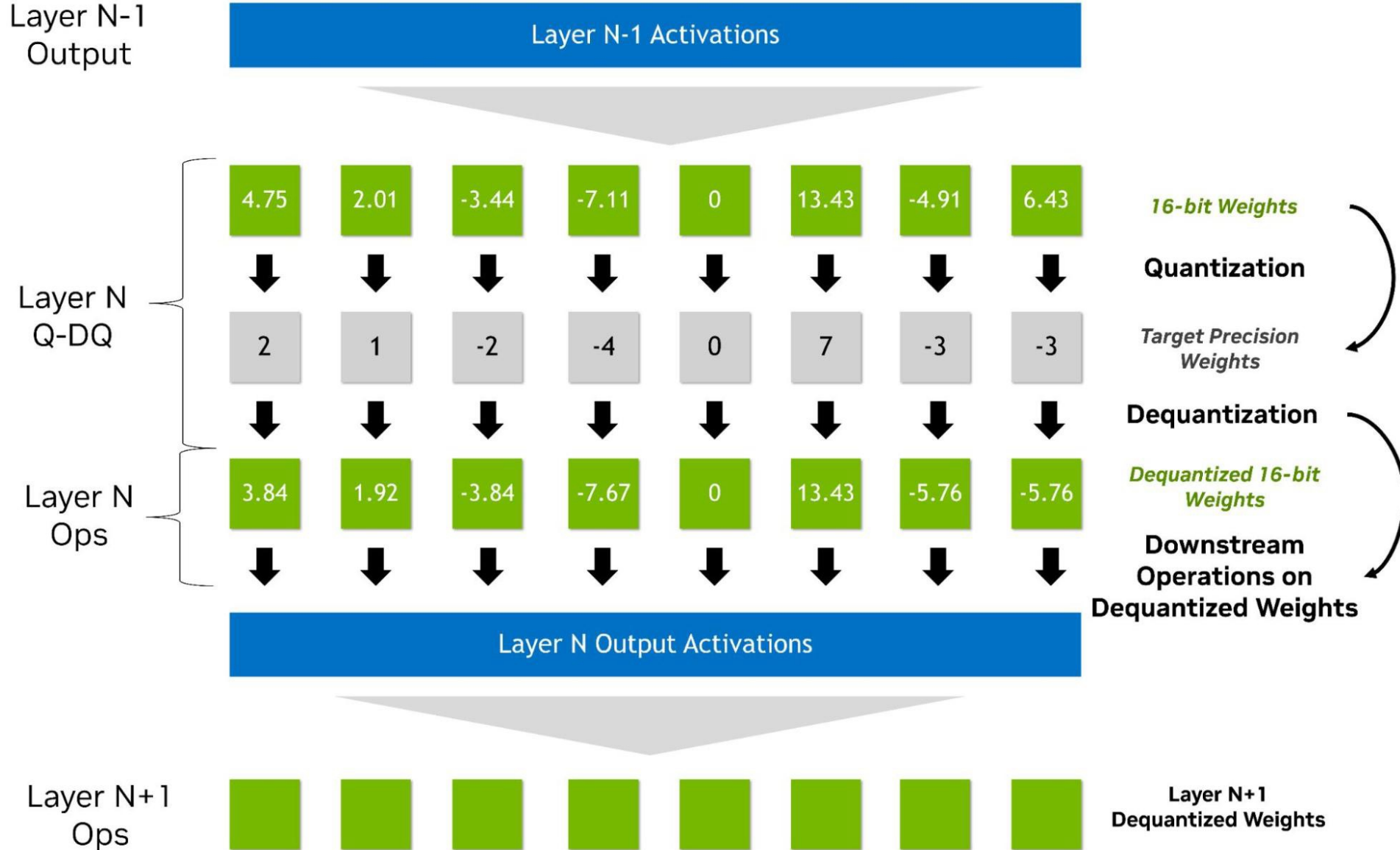
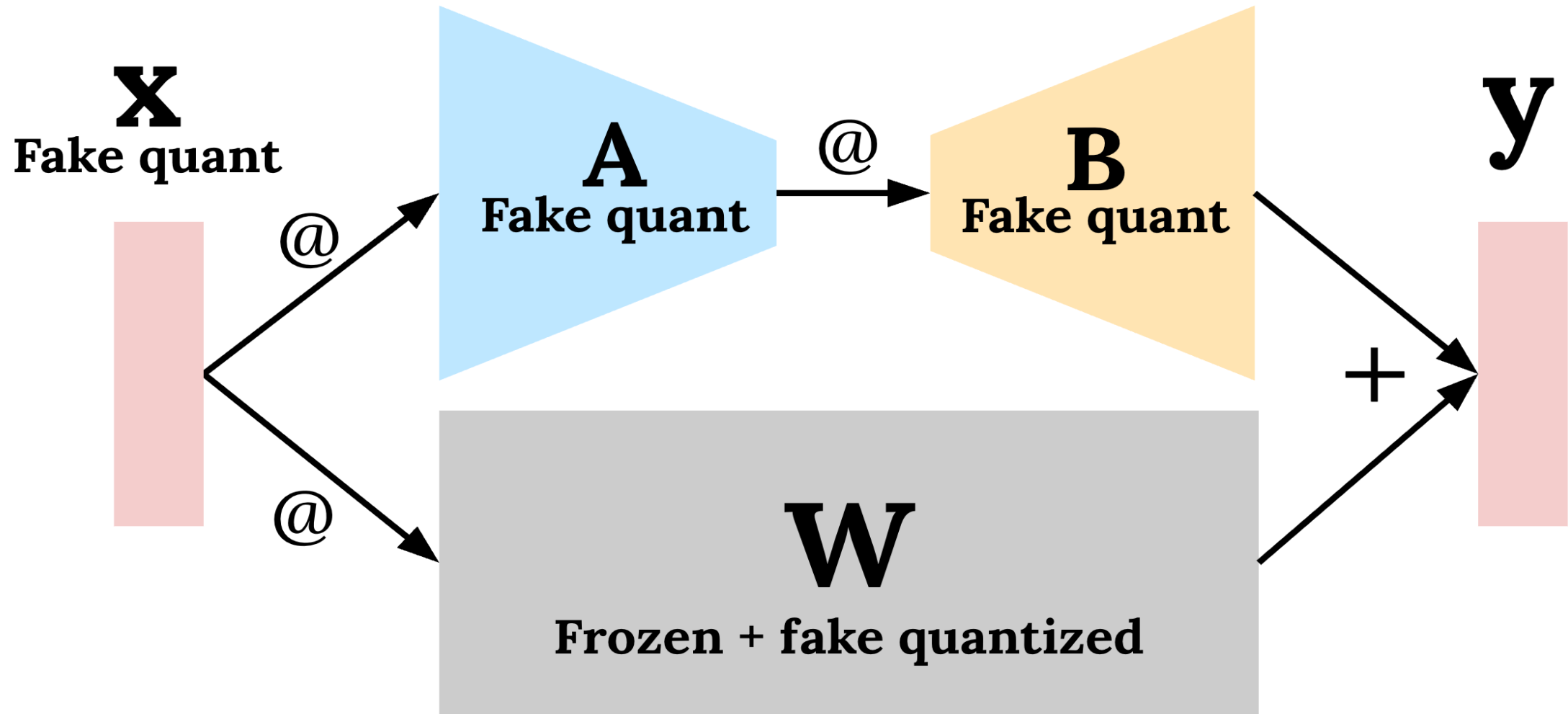


Diagram of a “Fake Quantization” on weights during QAT



Combining QAT with LoRA



Full Finetuning vs LoRA vs QLoRA vs QAT-LoRA

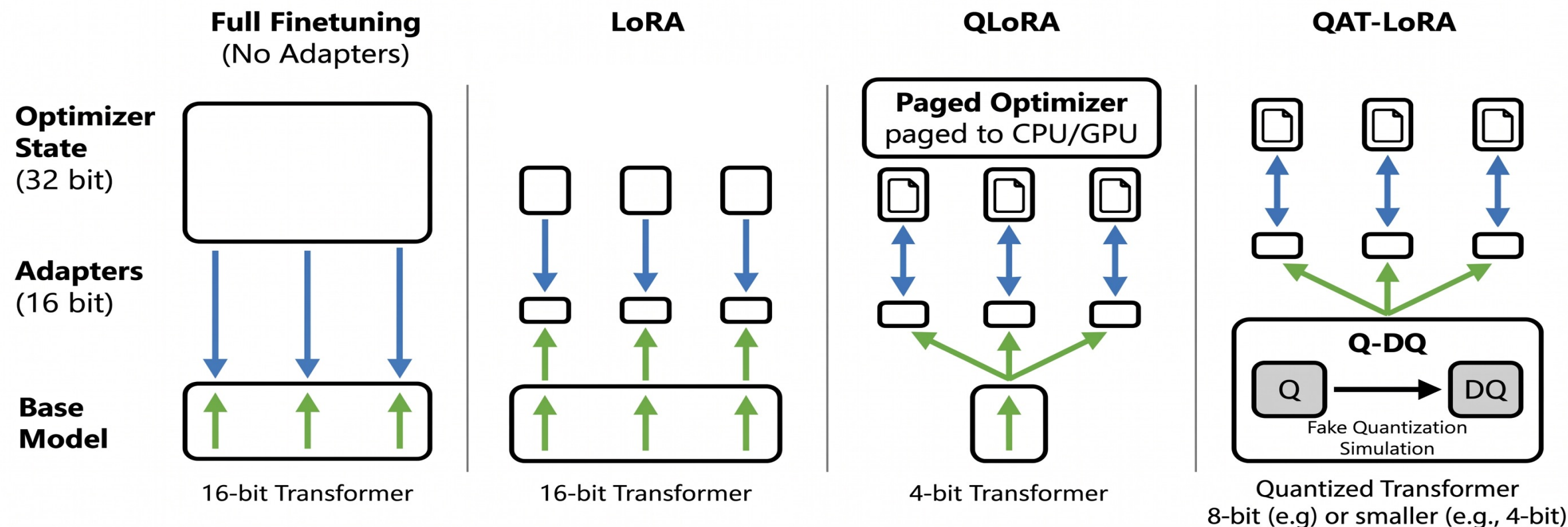
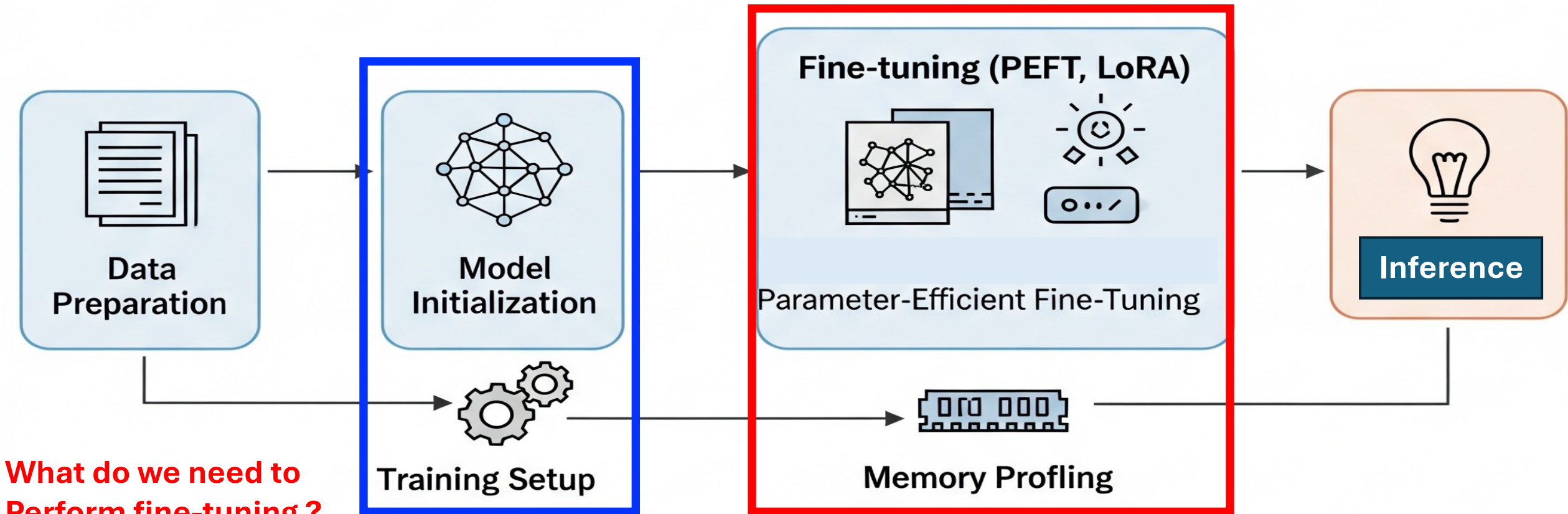


Figure 1: Different finetuning methods and their memory requirements. QLoRA and QAT-LoRA improve over LoRA by quantizing the transformer model to low precision; QAT-LoRA utilizes fake quantization simulation (Q-DQ) of the 4-bit base model during adapter training to mitigate precision loss, producing high-accuracy quantized models with minimal fine-tuning overhead, unlike standard QLoRA which only quantizes post-training.

Adapted <https://arxiv.org/abs/2305.14314>

Fine-tuning pipeline for LLM



What do we need to
Perform fine-tuning ?

Xsum dataset

Llama model

Config files (.yaml)

Recipes (python scripts)

Path

shared/datasets

shared/models

configs/

recipes/



State-of-the-art LLaMA

Modern LLMs are primarily based on the **Transformer** architecture (see <https://arxiv.org/pdf/1706.03762>)

LLaMA developed by **Meta AI** as an open-source alternative to proprietary LLMs e.g. GPT

State-of-the-art LLaMA:

Llama 4 models ⓘ

Text Multi-image New

Llama 4 Scout

- Superior text and visual intelligence
- Class-leading 10M context window
- **17B active params x 16 experts, 109B total params**
- Llama Guard 4 12B is included
- Llama Prompt Guard 2 22M and Llama Prompt Guard 2 86M are included

Text Multi-image New

Llama 4 Maverick

- Our most powerful open source multimodal model
- Industry-leading intelligence and fast responses at a low cost
- **17B active params x 128 experts, 400B total params**
- Llama Guard 4 12B is included
- Llama Prompt Guard 2 22M and Llama Prompt Guard 2 86M are included

Llama 3 models ⓘ

Text

Llama 3.3: 70B

- Multilingual open source large language model
- Experience 405B performance and quality at a fraction of the cost

*Licensed under Llama 3.3 Community License Agreement

Lightweight

Llama 3.2: 1B & 3B

- Lightweight and most cost-efficient models you can run anywhere on mobile and on edge devices
- Llama Guard 3 1B is included
- Quantized models available

*Licensed under Llama 3.2 Community License Agreement

Text

Llama 3.1: 405B & 8B

- Multilingual open source large language model
- Llama Guard 3 8B and Llama Prompt Guard 2 are included

Multimodal

Llama 3.2: 11B & 90B

- Open multimodal models that are flexible and can reason on high resolution images and output text
- Llama Guard 3 11B Vision is included

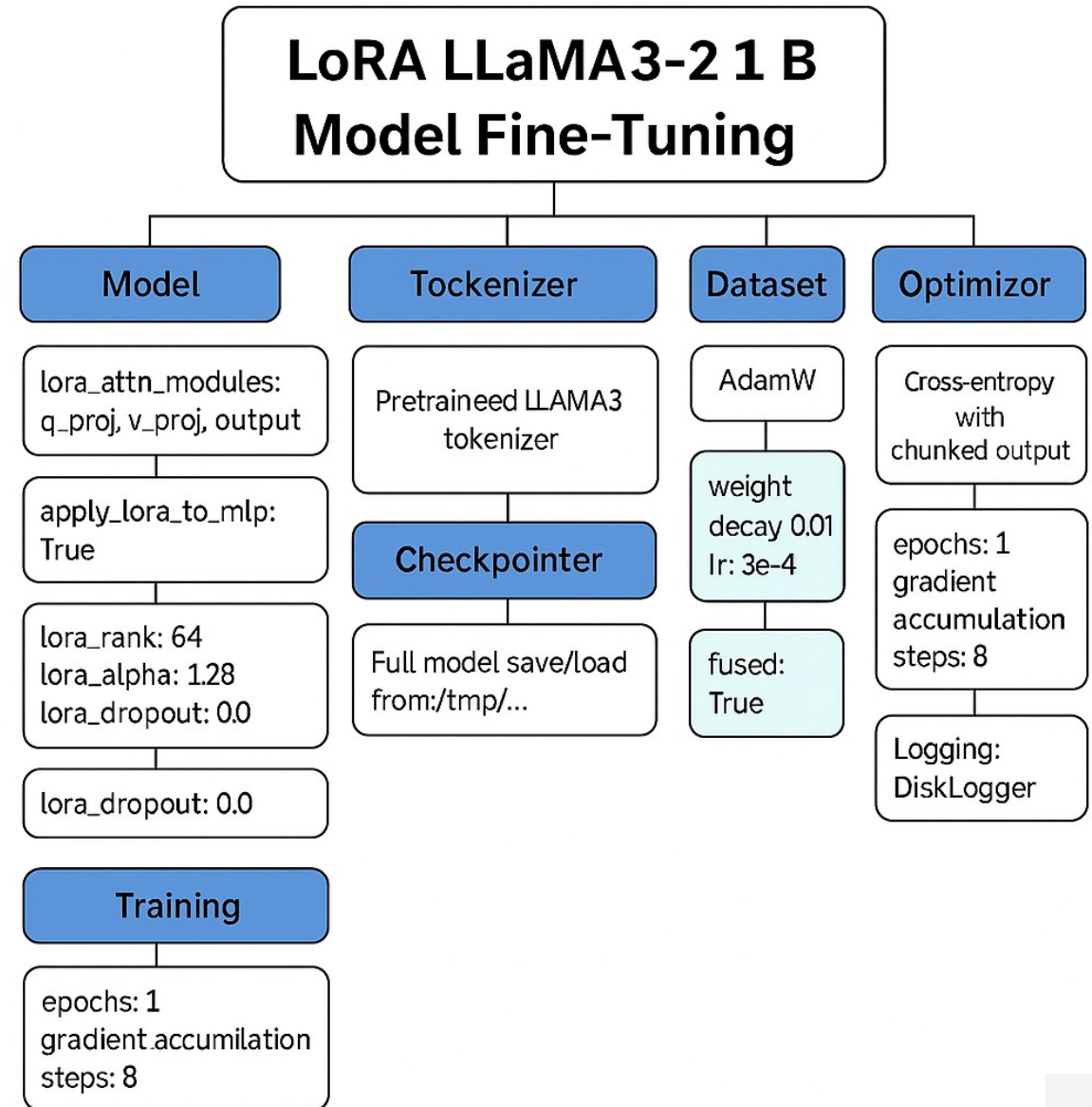


LLaMA3-2 1B Fine-Tuning Config Overview

Single GPU

Due to limited computational resources:

- Light model: 1B parameter of LLaMA3-2
- LoRA techniques for fine-tuning

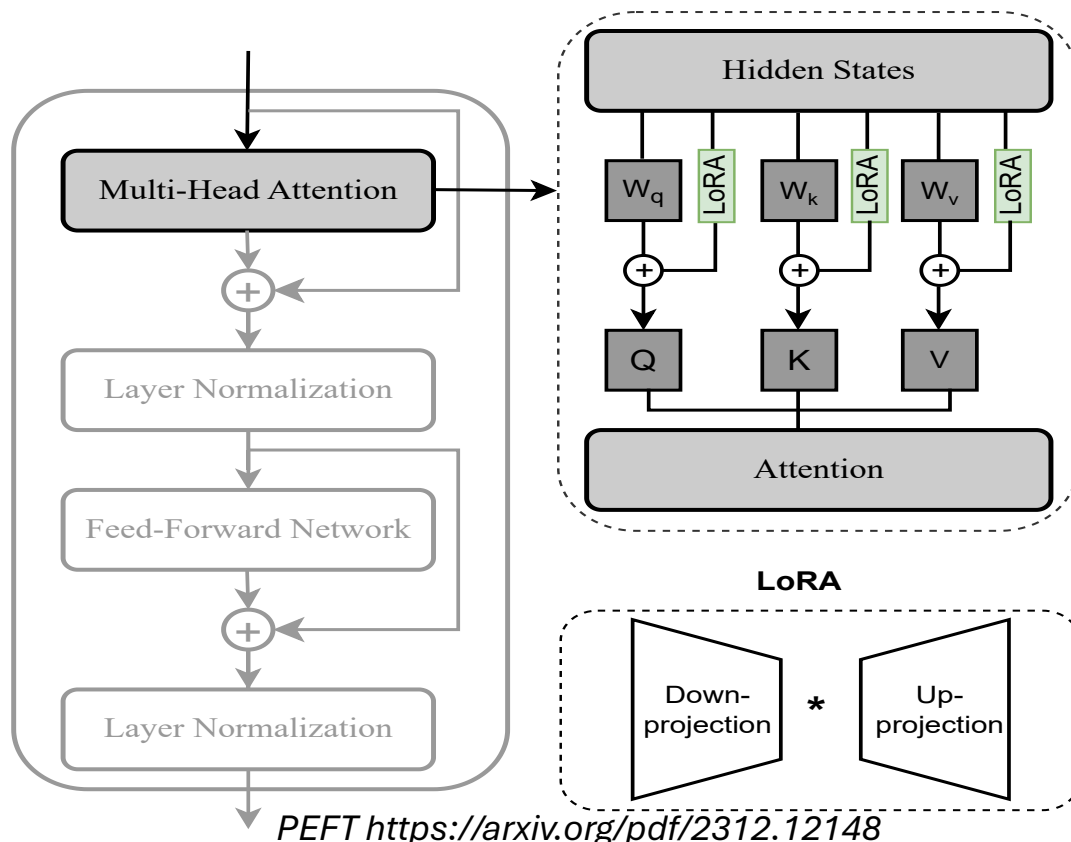


Model Arguments

```
model:
  _component_: torchtune.models.llama3_2.lora_llama3_2_1b
```

```
lora_attn_modules: ['q_proj', 'v_proj', 'output_proj']
```

- Loads the LLaMA3-2 1B model with **LoRA** integration. It's a **1B parameter variant of LLaMA3.2** (Meta's LLaMA3 architecture)
- LoRA is applied to specific transformer attention components: **query, value, and output projections**.



- The **Query vector** represents what information a token is trying to find.
- The **Key vector** represents what information the token contains.
- The **Value vector** carries the actual content that will be aggregated during attention.

Model Arguments

```
model:  
  _component_: torchtune.models.llama3_2.lora_llama3_2_1b
```

```
lora_attn_modules: ['q_proj', 'v_proj', 'output_proj']
```

```
apply_lora_to_mlp: True
```

```
lora_rank: 64  
lora_alpha: 128  
lora_dropout: 0.0
```

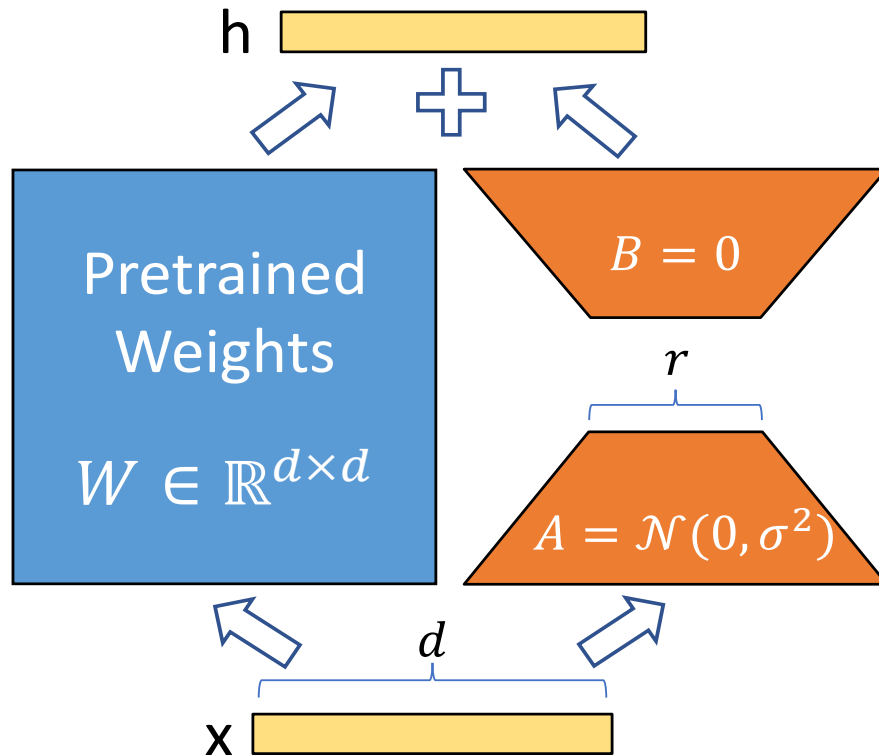
- Loads the LLaMA3-2 1B model with **LoRA** integration. It's a **1B parameter variant of LLaMA3.2** (Meta's LLaMA3 architecture)
- LoRA is applied to specific transformer attention components: **query, value, and output projections**.
- Enables **LoRA** for the model's MLP (feedforward) layers too, not just attention. (`apply_lora_to_mlp: False`: only to attention layers)
- **lora_rank** determines low-rank dimensionality of matrices **A** and **B** that are added to the pre-trained model.
- **lora_alpha** scales LoRA updates (commonly $2 \times \text{rank}$).
- **lora_dropout** is optional and is turned off (can be tuned if overfitting is noticed).



Model Arguments

```
apply_lora_to_mlp: True
```

```
lora_rank: 64  
lora_alpha: 128  
lora_dropout: 0.0
```



LoRA <https://arxiv.org/pdf/2106.09685>

- Enables **LoRA** for the model's MLP (feedforward) layers too, not just attention. (`apply_lora_to_mlp: False`: only to attention layers)
- **lora_rank** determines low-rank dimensionality of matrices **A** and **B** that are added to the pre-trained model.
- **lora_alpha** scales LoRA updates (commonly $2 \times \text{rank}$).
- **lora_dropout** is optional and is turned off (can be tuned if overfitting is noticed).

Modified forward pass

$$h = W_0 x + \Delta W x = W_0 x + \underbrace{B A x}_{\text{LoRA update}}$$

Random Gaussian initialization for A and zero for B

$\Delta \tilde{W} = BA$ is zero at the beginning of training



Tokenizer

```
tokenizer:
  _component_: torchtune.models.llama3.llama3_tokenizer
  path: /tmp/Llama-3.2-1B-Instruct/original/tokenizer.model
  max_seq_len: null
```

- Specifies a tokenizer built for LLaMA 3 models.
- Defines the path to the tokenizer model file.
- Specifies the maximum number of tokens the tokenizer should return
- **null** means no limit is enforced here.

Checkpoint

```
checkpointer:
  _component_: torchtune.training.FullModelHFCheckpointer

  checkpoint_dir: /tmp/Llama-3.2-1B-Instruct/
  checkpoint_files: [model.safetensors]
  recipe_checkpoint: null
  output_dir: /tmp/Llama-3.2-1B-Instruct/
  model_type: LLAMA3_2
  resume_from_checkpoint: False
  save_adapter_weights_only: True
```

- Class used for checkpointing.
- Saving and restoring the full Hugging Face model.
- Directory **where checkpoints are read from**.
- Tells the checkpointer **which specific files to load**.
- Loading a training recipe or configuration from a checkpoint.
- Directory to **save** the checkpoints.
- Specifies the model type Llama-3.2-1B-Instruct.
- **False**: Starts a new training from pre-trained base otherwise it continues from where it previously stopped.
- **False**: Entire model will be saved including LoRA weights.
- **True**: only **LoRA/adaptor weights** are saved.
(It will require a separate step to merge them later with the base model at inference).

Dataset & Sampler

Use API for the dataset

```
dataset:
  _component_: torchtune.datasets.alpaca_cleaned_dataset
  packed: False
seed: null
shuffle: True
batch_size: 4
```

- Dataset of instruction-following demonstrations based on OpenAI's text-davinci-003
- Indicates whether sequences are *packed* for training.
- **False:** Padding is used to ensure consistent sequence lengths within a batch.
- **True:** Input examples are packed together into a continuous stream, which can increase training speed by reducing padding overhead.

Use local dataset

```
dataset:
  _component_: torchtune.datasets.instruct_dataset
  source: json
  data_files: ${dataset_dir}
  split: train[:2000]
```

- No specific random seed is set.
- Dataset is shuffled to prevent the model from overfitting.
- Number of samples to process in **one step**.



Optimizer & Scheduler

optimizer:

```
_component_: torch.optim.AdamW  
fused: True  
weight_decay: 0.01  
lr: 3e-4
```

- Specifies the optimizer class: AdamW is widely used in Transformer training.
- Enables a **fused backend implementation of the optimizer** (speeds up training).
- Helps prevent overfitting.
- This is the initial learning rate.

lr_scheduler:

```
_component_: torchtune.training.lr_schedulers.get_cosine_schedule_with_warmup  
num_warmup_steps: 100
```

- Defines how the learning rate changes over time.
- Uses a built-in TorchTune utility to get a **cosine decay** schedule with warmup.
- During warmup, the LR gradually increases from zero to the target LR.
- After warmup, the LR follows a **cosine decay**, slowly decreasing over time (effective for transformer models)
- Number of steps to gradually increase the learning rate before cosine decay begins.

Loss Function

loss:

```
_component_: torchtune.modules.loss.CEWithChunkedOutputLoss
```

- Specifies the loss function used during training.
- **Cross-Entropy loss** designed to work with chunked model outputs.



Training Config

```
epochs: 1
max_steps_per_epoch: null
gradient_accumulation_steps: 8
compile: False
```

- Sets the total number of full passes over the training dataset.
- Sets the optional limit on how many steps to run **per epoch**.
- **Null:** means it will run over the **entire dataset** for each epoch.
- Sets how many steps before applying an optimizer update.
- Useful for a limited GPU memory. Number of forward + backward passes to accumulate gradients **before updating weights**.
- Allows simulating a **larger batch size** without increasing GPU memory usage.
- **False:** Disables torch.compile()
- **True:** Enables torch.compile(), which can speed up and optimize model execution.



Logging

```
output_dir: /tmp/lora_finetune_output
metric_logger:
  _component_: torchtune.training.metric_logging.DiskLogger
  log_dir: ${output_dir}
log_every_n_steps: 1
log_peak_memory_stats: True
```

- Specifies the directory where outputs (logs, metrics, etc.) will be saved
- **DiskLogger** writes logs to disk (Useful for later analysis or visualization).
- Sets the log directory to be the same as **output_dir**: /tmp/lora_finetune_output
- Logs metrics **after every single training step**.
- **True**: Enables logging of **peak GPU memory usage**.
- (useful for debugging memory issues or profiling training efficiency)

Environment

```
device: cuda
dtype: bfloat16
```

- Specifies that training will run on a **CUDA-compatible GPU**.
- Sets the computation **data type** to **bfloat16**, a lower-precision format.
- (16-bit floating-point format).
- Less precise than fp32 (32 bit) but more than fp16 (16 bit).



Activation Memory Control

```
enable_activation_checkpointing: False
enable_activation_offloading: False
```

Memory optimization Flags:

- **False:** Default. All activations checkpointings are stored (faster training, higher memory usage).
- Moves activations from **GPU to CPU** memory during training.
- **True:** Reduces GPU memory usage, but slows training. Useful for **very large models** that don't fit entirely on GPU.

Profiler Configuration

```
profiler:
  _component_: torchtune.training.setup_torch_profiler
  enabled: False
```

- Profile training performance (CPU/GPU utilization, memory, bottlenecks).
- False: means no profiling occurs.

```
output_dir: ${output_dir}/profiling_outputs # Saves traces (e
```

```
cpu: True # Profile CPU operations
cuda: True # Profile GPU (CUDA) operations
```

- Specifies the directory where profiling outputs are saved.
- Enables tracking of operations on CPU & GPU devices.



Hands-on:

LoRA Fine-Tuning on a single GPU

Dataset: Xsum & alpaca dataset

Summarization tasks

QA tasks

 **HichamAgueny** update project nbr

04ce371 · yesterday

 History

Name	Last commit message	Last commit date
 ..		
 finetuning	update project nbr	yesterday
 profiling	update project nbr	yesterday
 README.md	update readme	5 days ago

README.md

Day 1: Single GPU Workflow

This directory contains labs and scripts for performing LLM fine-tuning and performance profiling on a single GPU.